

141. Linked List Cycle

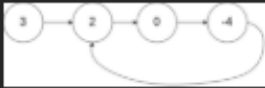
Easy Topics Compare

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. Note that `pos` is not passed as a parameter.

Return `true` if there is a cycle in the linked list. Otherwise, return `false`.

Example 1:



Inputs: `head = [3,2,0,-4]`, `pos = 1`

Output: `true`

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:



Inputs: `head = [1,2]`, `pos = 0`

Output: `true`

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Inputs: `head = [1]`, `pos = -1`

Output: `false`

Explanation: There is no cycle in the linked list.

</> Code

C Auto

```

1  /**
2   * Definition for singly-linked list.
3   * struct ListNode {
4   *     int val;
5   *     struct ListNode *next;
6   * };
7   */
8  bool hasCycle(struct ListNode *head) {
9      struct ListNode * temp =head;
10     struct ListNode * curr=head;
11
12     while(curr !=NULL && curr->next!=NULL){
13         temp=temp->next;
14         curr=curr->next->next;
15
16         if(temp==curr){
17             return true;
18         }
19     }
20     return false;
21 }
```

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 3 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

head =
[3,2,0,-4]

pos =
1

Output

true

Expected

true

☒ Testcase | [>_ Test Result](#)

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 3 ms

☒ Case 1 ☒ Case 2

Input

head =
[1,2]

pos =
0

Output

true

Expected

true

Accepted Runtime: 3 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

head =
[1]

pos =
-1

Output

false

Expected

false